Jerk-Limited Motion in `tpnext`

A reading note on `grandixximo/linuxcnc` branch `tpnext-ma`

Notes for a mill, not a patch.
Pinned to `0571a030` (7 Oct 2026)

7 October 2026

Contents

1 How this relates to the jerksplan writeup **1**

2 The object of planning **2**

3 One-dimensional kinematics **2**
3.1 Rest-to-rest ramps 2
3.2 Braking distance 3

4 Projecting axis and joint limits **3**
4.1 Joints 4
4.2 Jerk the cycle time can shape 4

5 Quintic blends (G64 P and E) **4**

6 Backward envelope **4**

7 The per-cycle controller **5**

8 Override, hold, abort **5**

9 Reverse run **6**

10 Spindle sync **6**

11 What this planner is not **6**

12 Mapping from the jerksplan sections **7**

13 Open questions (the useful kind) **7**

This is not the jerksplan PDF rewritten. That document repairs planner 1 in place. ***`tpnext`*** *throws the segment queue away. The 1-D identities are the same; the object being planned is not. Equations below are from the code, not from a wish list. Line numbers are of this pin.*

1 How this relates to the jerksplan writeup

The August 2026 spec (*Jerk-Limited Trajectory Planning in LinuxCNC*, apogee / `automata/jerksplan`) is a repair manual for the stock trajectory planner. Its type 2 is a clone of type 1: still a queue of TCs, still Ruckig/`cruckig` as a 1-DoF BVP solver, still "accel = 0 at joins," still geometry

entering only as scalar (v, a, j) caps. The work is to close holes (T-curve fallback, reverse-run sign, parabolic $2j_{\max}$, one-cycle kinks) without rewriting the kernel. Spindle position sync is *deliberately* left trapezoidal.

`tpnext` is the rewrite that document's author (Luca / grandixximo) started when he decided type 1 could not be patched into correctness. Enable it with

```
[TRAJ]
TPMOD = tpnextmod
PLANNER_TYPE = 1
MAX_LINEAR_JERK = ...
```

It ignores `ARC_BLEND_*`. Homing still does not use it (§11).

The physical requirements in the jerksplan PDF—override, reverse, sync, normal jerk, joints—still apply. Almost every *mechanism* is different. Section 12 is a section-by-section map.

2 The object of planning

`tpnext` plans one path parameter s along a nine-axis pose $P(s) \in \mathbb{R}^9$ (XYZABCUVW). State each servo cycle is (s, v, a, j) with $v = \dot{s}$, $a = \ddot{s}$, $j = \dddot{s}$ (`tpn.h:1-10`, `tpn.cur_s` et al.).

A queued *move* is a line or an arc in that pose space. A *corner* is not a join of two 1-DoF profiles. It is replaced, when G64 allows, by a quintic blend of half-length h on each side. The controller never sees "segment 7 of 40." It sees a chain of *pieces*:

- $N_{\text{sub}} = 6$ equal parts of the incoming blend (`TPN_NSUB`), so only the sharpest part of the corner runs at the lowest speed;
- then $n_{\text{int}} \leq 16$ parts of the unblended interior (finer where joints peak, e.g. near a singularity).

Each piece carries tangential limits (V, A, J) in path units (`tpn_lim`), plus a second set `lim_*_hi` opened toward the highest feed override.

Point-to-point joint moves (G53.4–G53.7) are a side door: s is joint space distance on a straight line in q; they stop at both ends and are not run backwards.

3 One-dimensional kinematics

Within a cycle of constant jerk j, the usual

$$a(t) = a_0 + jt, \tag{1}$$
$$v(t) = v_0 + a_0 t + \tfrac{1}{2} j t^2, \tag{2}$$
$$s(t) = s_0 + v_0 t + \tfrac{1}{2} a_0 t^2 + \tfrac{1}{6} j t^3. \tag{3}$$

There is **no Ruckig**. There is no 7-phase BVP. The runtime picks a single j for this period (§7) and integrates (1)–(3). Failed solves cannot fall back to a T-curve because there is no solver.

3.1 Rest-to-rest ramps

A speed change Δv with $a = 0$ at both ends, under bounds A, J (`tpnRampTime`, `tpn_geom.c:618-627`):

$$T(\Delta v) = \begin{cases} 2\sqrt{\Delta v / J} & \Delta v \cdot J \leq A^2 \quad \text{(triangular)} \\ \Delta v / A + A / J & \text{otherwise (holds } A\text{).} \end{cases} \tag{4}$$

Distance is the trapezoid area $D = \frac{1}{2}(v_0 + v_1)T$.

The inverse—how much speed a ramp from V with $a = 0$ can pick up over distance D—is `tpnRampReach`. For the triangular case it solves

$$x^3 + 2Vx = D\sqrt{J}, \qquad \Delta v = x^2 \tag{5}$$

by Cardano (one real root); past $\Delta v = A^2/J$ it is a quadratic in Δv. From rest this is $\Delta v = (D^2 J)^{1/3}$ until $DJ^2 > A^3$.

3.2 Braking distance

`tpnBrakeDist`(v_0, a_0, v_t, A, J): shortest distance in which the state can be brought to a speed no higher than v_t with $a = 0$ at the end. If $a_0 > 0$, first ramp $a \to 0$ at $-J$; then a symmetric $T(\Delta v)$ down to v_t. If already decelerating, it continues a profile that started at $(v_*, 0)$ and ramped down to a_0. Zero if the state never exceeds v_t.

Lookahead and the runtime both use this with a brake scale

$$A_b = 0.97\,A, \qquad J_b = 0.97\,J \qquad (\texttt{TPN_BRAKE_SCALE}). \tag{6}$$

The 3% is headroom so the cycle-by-cycle picker can follow the curve without saturating the true limits.

4 Projecting axis and joint limits

This is the actual jerk algorithm. Geometry is not a post-filter. For $x_k(s) = P_k(s)$,

$$\dot{x}_k = v\,P_k', \tag{7}$$

$$\ddot{x}_k = a\,P_k' + v^2 P_k'', \tag{8}$$

$$\dddot{x}_k = j\,P_k' + 3va\,P_k'' + v^3 P_k'''. \tag{9}$$

Write $G_k = |P_k'|$, $G1_k = |P_k''|$, $G2_k = |P_k'''|$ (bounds, not point samples: lines are constant; blends use the Bernstein hull of the derivatives, `tpnBlendBounds`).

On a *straight* piece $P'' = P''' = 0$, so $V \le v_k/G_k$, $A \le a_k/G_k$, $J \le j_k/G_k$: the characteristic-time projection of jerksplan (38), exact.

On a *curved* piece the v^2P'' and v^3P''' terms spend acceleration and jerk even at constant v. The code reserves half of A and a quarter of J for those terms, or more if the programmed feed needs it (`tpn_geom.c:428-477`):

$$f_a = \tfrac{1}{2}, \quad f_j = \tfrac{1}{4}, \tag{10}$$

$$V_2 = \sqrt{f_a\,a_k/G1_k}, \qquad V_3 = \sqrt[3]{f_j\,j_k/G2_k}, \tag{11}$$

$$V_{2,\max} = \sqrt{0.8\,a_k/G1_k}, \qquad V_{3,\max} = \sqrt[3]{0.6\,j_k/G2_k}. \tag{12}$$

The speed cap of the piece is $\min(V_2, V_3)$, opened toward the feed up to the max forms so a chain of arcs does not hunt. Remaining budget at that speed V (`tpnLimitsAt`):

$$A = \min_k \frac{a_k - V^2 G1_k}{G_k}, \tag{13}$$

$$J = \min_k \frac{\frac{2}{3}(j_k - V^3 G2_k)}{G_k}, \tag{14}$$

with the leftover third of jerk also bounding the $3vaP''$ cross term. That is the jerksplan §8 "jerk ratio r_j" idea, derived from the chain rule instead of Frenet-on-circles.

4.1 Joints

For non-identity kinematics the same projection is applied to $q(s)$ with bounds

$$|q'| \le G, \quad |q''| \le r\,G1_s + G1_u, \quad |q'''| \le r^2 G2_s + rG2_m + G2_u$$

(`tpn_jb`; r is the blend scale). This is the opposite of jerksplan §15 ("joint jerk is monitored, not enforced"). Near a singularity a joint that can only follow the path below `SINGULAR_FLOOR` of feed either stops the move or runs at that floor (`SINGULAR_MODE`).

4.2 Jerk the cycle time can shape

When limits are read (`tpn_plan.c:31-48`), every axis jerk is capped at

$$j_{\text{trap}} = A/(2T_{\text{servo}}). \tag{15}$$

Faster than that cannot be shaped in two cycles; it is the inherent jerk of a T-curve. Position-synced moves (§10) take this trapezoid jerk regardless of `PLANNER_TYPE`: a lower J only delays catch-up on a thread. All INI limits are further scaled by 0.99 (`TPN_LIMIT_SCALE`).

5 Quintic blends (G64 P and E)

A blend of half-length H is a quintic Hermite from (P_0, P_0', P_0'') to (P_1, P_1', P_1'') in $\tau = \sigma/H \in [0,1]$ (`tpnBlendInit`):

$$P(\tau) = \sum_{n=0}^{5} c_n \tau^n, \tag{16}$$

with the usual C^2 coefficients $c_0 = P_0$, $c_1 = HP_0'$, $c_2 = \frac{1}{2}H^2P_0''$, and c_3, c_4, c_5 matching position, first and second derivatives at $\tau = 1$. This *is* the transition window jerksplan proposed as a time τ_{kink}. Here it is a length H inside the tolerance tube, not a smear of Δa over N servo periods.

G64 **P** weights linear axes, **E** weights angular axes, as one tube (`joinMoves`). Deviation is the weighted distance from the programmed corner to the blend; H is reduced until that distance is ≤ 0.98 of the tolerance (`TPN_TOL_SCALE`). Between two lines the blend only scales about the corner, so one shot suffices.

A longer blend is faster at the apex (jerk-limited speed grows about as $H^{1/3}$) but takes longer to cross. The planner tries H, $H/2$, … (6 halvings, down to about 4 cycles across the blend) and **a stop**, and keeps the fastest the controller can still reach. Sharp 90° corners therefore often *stop* even in G64 if a fillet would be slower. That is intentional, not a failure of jerk limiting.

`ARC_BLEND_*` is documented as tuning `tpmod` only.

G64 **R** is a corner-speed scale (and scales `SPEED_HUMP_TIME`). It is not a second planner.

6 Backward envelope

Lookahead is not `findSCurveMaxStartSpeed` on the next TC. Each piece stores a *ramp* (`tpn_ramp`):

> down to speed E at path S, with no acceleration left, under limits A, J, and below the smallest speed cap V of the pieces it crosses. $E < 0$ means "to rest at the end of the queue, wherever that is when you read it."

The backward pass keeps up to three ramps besides the one from the next piece, re-evaluated every 5% of the longest (`tpn_plan.c:86-211`). Where a piece takes its own cap, older ramps die. Braking uses (6).

This is the fix for the lie the earlier backward pass told: it used to promise speeds with half of each piece's A and **no** J, so the controller discovered too late that it could not dump speed (short corners, 3-D chips at 100–300× jerk). The envelope is now a real S-curve brake.

The ramp to rest at the *end of the queue* used to be rewritten on every add—at low J it reaches back through every piece. As of this pin it is stored as "to the end, with the limits of that build," and evaluated at runtime from where the queue actually ends (`tpnEndRamp`, at most 32 later generations). Adds on a dense file dropped from thousands of rewrites to tens.

`SPEED_HUMP_TIME`: a speed-up that would have to brake again inside that window is flattened. Short CAM facets then do not buzz.

7 The per-cycle controller

`tpn_run.c` header: pick the largest jerk that keeps every speed cap and stop in the braking horizon reachable, and land stops on a cycle boundary.

Each cycle (`tpnAdvance`):

1. `gather` (or `gatherReverse`) the pieces ahead into up to 128 constraints: hard cap V_h, optional soft (requested) speed V_s, A/J on the way and from that point on, and the envelope ramp.
2. `chooseJerk`: admissible $j \in [-J, J]$ also clipped so $|a + jT| \le A$. Raise the lower bound until the step cannot reverse. If the most aggressive j already satisfies every constraint, take it. Otherwise bisect (18 steps) the largest j that does. Soft caps that cannot be met even braking full-rate are *tracked*, not searched (§8). If nothing meets the hard set (a hair past a tight cap), keep speeds and dump a as fast as they allow.
3. Optional: flatten a short hump; follow the spindle (§10); `landJerk` onto a speed plateau without chattering j sign; `finishStop` deadbeat onto a rest on a cycle boundary.
4. Integrate. Do *not* snap onto a stop a cycle early—that dumped leftover deceleration in one period (1.5–2.5× joint jerk at 125 μs before the deadbeat-finish commit).

A constraint is feasible if `tpnBrakeDist` to its cap fits in the distance left, or if the motion is already on a ramp that *carries* through that boundary still braking (`conArriveOk`, `carriers`). That is how you may enter a slower piece above its envelope: you are still on a legal S-curve down to a later E.

There is no T-curve fallback. If the picker cannot honour J, it still picks a j; the contract is "best effort under the gathered ramps," not "call Ruckig and if `Working` fails, bang A."

8 Override, hold, abort

Jerksplan keeps the queue-time plan at $f_{\max}$ and absorbs override as a Ruckig replan from (s, v, a). `tpnext` stores two cap sets so a higher override can open curvature caps without rebuilding. A *lower* override is a soft cap. Tracking it ramps deceleration out with the **lowest J of the pieces the ramp will cross**, not the J of the piece you are in (`jerkAhead`). Using the local J on a short high-limit piece, then entering a lower-J piece, ran speed to zero mid-path at up to 9× jerk (`sine_test` 1.8 → 0.3). Requested stop (feed hold, pause, abort) already did the look-ahead; override tracking now matches.

Paused at rest: hold still. Searching for a tiny j was creeping.

Feed override while moving, pause, abort, and 125 μs–1 ms servo periods are the tests this week was written for.

9 Reverse run

History of 200 completed moves. Reverse is along $-s$, same controller, same envelopes replayed (`tpnEnvBack`). Synced, rigid-tap, indexer, and point-to-point joint moves refuse reverse. This is not jerksplan's reverse (stop-and-go at every TC boundary because v_{final} is forced to zero). Whether reverse *blends* as well as forward is still a field question; the machinery is the path parameter, not a second optimiser.

10 Spindle sync

Opposite decision from jerksplan §13.

Jerksplan type 2 **suspends** jerk on G33/G33.1 because a third-order tracker lags a second-order one, and lag is pitch error. Measured $\sim 2.8\times$ worse pitch under planner 1's jerk-limited sync on a spindle disturbance.

`tpnext` keeps jerk on. A synchronized move starts on the index (or D revolutions past it). The lag of the jerk-limited start is not dropped: a catch-up planned on the index gains it back, so the thread lies in the same place at any spindle speed. D waits at rest past the index, then starts as on the index; rigid tap ignores D (one cut). Per-cycle tracking is feedforward jerk plus three poles at $\omega = 80\,\text{rad/s}$ (`followJerk`), with 90% of A and 80% of J left as catch-up margin. Far from the reference it bisects the j that can still land, the same idea as a stop.

Whether that wins on a real spindle vs jerksplan's trapezoid-in-the-thread is an empirical fight, not a settled theorem. The offline test `spindle-angle-offset` already sees the start a hair later than `tpmod` because Z does not jump 0.02 mm in one cycle.

11 What this planner is not

Jog and homing still use `simple_tp`, a different 7-segment S-curve. That is why you home with `PLANNER_TYPE` $= 0$ and switch type 1 from HAL afterwards. Homing distances were sized for $v^2/2A$. Finite jerk on a short search/latch overshoots the switch into the hard stop. Pendant `ilowpass` is a third path.

Normal jerk $|d\vec{a}/dt|$ is not the 1-D j. Caps (11) bound the speed-dependent terms of the chain rule; they do not bound the vector sum the way a true $|j| \le j_{\max}$ on $\mathbb{R}^3$ would. Quintics are C^2 in P if the end accelerations match; they are not clothoids (constant sharpness). Grotius's `tpmod_scurve` is that other geometry.

The interpolator is bang-bang in j. Industry OTGs (Ruckig, Reflexxes) solve a full 7-phase profile each period. `tpnext` picks $j \in \{-J, \ldots, +J\}$ for *this* T so the horizon stays feasible. Over many cycles that traces an S-curve. It is not the same object as "Ruckig on s."

12 Mapping from the jerksplan sections

Jerksplan	`tpnext`
§2 1-D / Ruckig	§3: same polynomials, no Ruckig, no T-curve fallback
§3–4 lookahead queries	§6: envelope of ramps, not `MaxStartSpeed`
§5 override replan	§8: soft cap + `jerkAhead`
§6 reverse mapping	reverse along s, 200-move history
§7–8 Frenet caps	§4: chain rule V_2, V_3, leftover A, J
§9 blend-arc / s_1 6× bug	§5: quintic in P/E tube; no s_1
§10–11 kink windows τ	quintic *is* the window (space, not time)
§12 parabolic $2j_{\max}$	no parabolic overlap
§13 sync stays trapezoid	§10: jerk on; D-word; opposite bet
§15 joints monitored	Jacobian limits on every piece
§16 `simple_tp`	unchanged; home in type 0
§17 clothoids / κ'	not here; quintic C^2, not G2 clothoid

13 Open questions (the useful kind)

1. Is 0.97 brake scale enough on a stiff mill at 1 ms, and too much lag at 125 μs?
2. Curvature budget $f_a = \frac{1}{2}$, $f_j = \frac{1}{4}$ (open to 0.8 / 0.6 toward feed): does a helix still trip FERROR before A on the tangent?
3. G64 "fastest of fillet vs stop": does a 90° finishing pass want that, or a forced fillet even if slower?
4. Sync: pitch error vs smoothness on a real encoder. Jerksplan has a number ($\sim 2.8\times$). `tpnext` does not, yet.
5. Jog/home on the same law as coordinated s. Until then, type 0 homing is not optional on hardware.
6. Vector $|d\vec{a}/dt|$ vs tangential J. Caps bound components. A square with G61 should S-ramp the sides and nearly stop; a FERROR spike at the corner is a failed envelope, not "jerk limiting."

How to read a test against this

Same g-code, three ways: `tpmod` type 0, `tpmod` type 1, `tpnextmod` type 1. Scope commanded v, a and FERROR. Type 1 should show S-shaped a, not square pulses (square = T-curve somewhere). Short polylines should hold feed if the envelope is doing its job. Override 100% → 30% while moving should not collapse speed to zero. Home in type 0. Air-cut.

Sources: `src/emc/tpnext/tpn.h`, `tpn_geom.c`, `tpn_plan.c`, `tpn_run.c`, `tpn_sync.c` on `grandixximo/linuxcnc tpnext-ma` at `0571a030`. Commit messages of 5–7 Oct 2026 for envelope, override, deadbeat, and D-word.